7zpus.swe@gmail.com

Specifica Tecnica

Tabella di Versionamento

Versione	Data	Autore	Verificatore	Descrizione
0.1.0	2026/02/28	Laoud Zakaria	Georgescu Diana	Creazione del documento e stesura struttura iniziale

Indice

1 Introduzione **5**
1.1 Obiettivo del documento . . . 5
1.2 Glossario . . . 5
1.3 Riferimenti . . . 5
1.3.1 Riferimenti normativi . . . 5
1.3.2 Riferimenti informativi . . . 5

2 Tecnologie utilizzate **7**

3 Linguaggio di Programmazione: TypeScript **7**
3.1 Deployment: Electron . . . 7
3.2 Framework Frontend: Angular . . . 8
3.3 Librerie Aggiuntive . . . 8
3.4 Tecnologie di analisi statica . . . 9
3.5 Tecnologie di analisi dinamica . . . 9

4 Contesto del sistema **9**
4.1 Modello architetturale . . . 10
4.2 Descrizione degli attori esterni . . . 10

5 Architettura dei container **11**
5.1 Renderer . . . 12
5.1.1 Struttura architetturale . . . 12
5.2 Main . . . 12
5.2.1 Ruolo architetturale . . . 12
5.2.2 Stack tecnologico . . . 12
5.2.3 Layer architetturali . . . 13
5.2.4 Sequenza runtime generale . . . 13
5.2.5 Dependency Injection . . . 13
5.2.6 Bootstrap e indicizzazione . . . 14
5.2.7 Wiring runtime e registrazioni . . . 14
5.2.8 Semantica transazionale condivisa . . . 14
5.2.9 Sicurezza preload . . . 14

6 Architettura delle componenti **15**
6.1 [Componente 1] . . . 15
6.1.1 Descrizione . . . 15
6.1.2 Interfacce . . . 15
6.1.3 Dipendenze . . . 15
6.1.4 Tracciamento dei requisiti . . . 15
6.2 [Componente 2] . . . 15
6.2.1 Descrizione . . . 15
6.2.2 Interfacce . . . 15

6.2.3 Dipendenze 15
6.2.4 Tracciamento dei requisiti 15
6.3 Componenti di Ricerca (FrontEnd) 16
6.3.1 Descrizione 16

7 Backend - Browsing 17
7.1 Panoramica 17
7.1.1 Organizzazione file backend 17
7.1.2 Flusso end-to-end 17
7.1.3 Use Case: firme getter 18
7.1.4 Layer Repository 19
7.1.5 Layer DAO 20
7.1.6 Entity e Value Objects 20
7.1.7 Data Transfer Objects 20
7.1.8 Mapping e conversioni 21
7.1.9 Comportamenti runtime 21
7.1.10 Test coverage 22

8 Backend - Verifica Integrita 23
8.1 Panoramica 23
8.1.1 Perimetro funzionale 23
8.1.2 Organizzazione file backend 23
8.1.3 Flusso end-to-end 24
8.1.4 Use Case: firme check 24
8.1.5 Servizio centrale 25
8.1.6 Regole di aggregazione 26
8.1.7 Hashing e transazioni nel modulo 26
8.1.8 Layer Repository e DAO 27
8.1.9 Comportamenti runtime 27
8.1.10 Test coverage 28

Elenco delle figure

1 Diagramma C1 - Contesto del sistema *DIPReader* 10
2 Diagramma C2 - Container del sistema *DIPReader* 11

Elenco delle tabelle

2 Dettagli sulla scelta di TypeScript come linguaggio di programmazione . 7
4 Dettagli sulla scelta di Electron come piattaforma di deployment 7
6 Dettagli sulla scelta di Angular come framework frontend 8
8 Dettagli sulle scelte di librerie aggiuntive 8
10 Dettagli sulle scelte di tecnologie di analisi statica 9
12 Dettagli sulle scelte di tecnologie di analisi dinamica 9
15 Contratto IPC del modulo Browsing 18
21 DTO schema (metadati caricati ricorsivamente) 21
25 Organizzazione modulo Verifica Integrita 23
26 Contratto IPC del modulo Verifica Integrita 24
28 Firme use case Verifica Integrita 25
30 Semantica operativa per livello 26
32 Repository e DAO coinvolti nella verifica integrita 27
34 Copertura test modulo Verifica Integrita 28

1 Introduzione

1.1 Obiettivo del documento

L'obiettivo del presente documento è descrivere in modo rigoroso le scelte tecnologiche e l'architettura del prodotto *DIPReader*, adottando il modello di progettazione *C4*.
L'analisi verrà condotta partendo dalla descrizione del contesto applicativo, per poi approfondire la struttura dei container, delle componenti e, infine, del codice, al fine di fornire una rappresentazione completa, coerente e sistematica del sistema.

1.2 Glossario

Ogni termine tecnico o con un significato particolare nell'ambito dell'Ingegneria del Software, utilizzato nella documentazione di progetto, è definito nell'apposito documento Glossario (v2.0.0).

1.3 Riferimenti

1.3.1 Riferimenti normativi

- Norme di Progetto v2.0.0 (v2.0.0)
- Standard SDD/IEEE/1016:2009(ultimo accesso: 02/03/2026)
- Regolamento di Progetto Didattico a.a. 2025/2026 (ultimo accesso: 02/03/2026)

1.3.2 Riferimenti informativi

- Capitolato C3: DIPReader (ultimo accesso: 10/02/2026)
- Dispense su Dependency Injection (Ultimo accesso: 02/03/2026)
- Dispense su OOP (Ultimo accesso: 02/03/2026)
- Dispense su Diagrammi delle classi (Ultimo accesso: 02/03/2026)
- Dispense su Pattern architetturali (Ultimo accesso: 02/03/2026)
- Dispense su Pattern creazionali (Ultimo accesso: 02/03/2026)
- Dispense su Pattern strutturali (Ultimo accesso: 02/03/2026)
- Dispense su Principi SOLID (Ultimo accesso: 02/03/2026)
- Dispense sulla Progettazione software (Ultimo accesso: 02/03/2026)
- Dispense sulla Qualità del software (Ultimo accesso: 02/03/2026)
- Repository Architettura esagonale (Kotlin) (Ultimo accesso: 02/03/2026)

- Repository Architettura esagonale (Java) (Ultimo accesso: 02/03/2026)
- Repository Ingegneria del software - swe-imdb (Ultimo accesso: 02/03/2026)
- C4 Model (Ultimo accesso: 03/03/2026)
- PlantUML (Ultimo accesso: 03/03/2026)
- Riferimenti a scelte tecnologiche:
  - TypeScript (Ultimo accesso: 03/03/2026)
  - Angular (Ultimo accesso: 03/03/2026)
  - Electron (Ultimo accesso: 03/03/2026)
  - SQLite (Ultimo accesso: 03/03/2026)
  - sqlite-vss (Ultimo accesso: 03/03/2026)
  - ONNX Runtime (Ultimo accesso: 03/03/2026)
  - Transformers.js (Ultimo accesso: 03/03/2026)
  - Node.js (Ultimo accesso: 03/03/2026)
- Glossario (v2.0.0)
- Piano di Progetto (v2.0.0)
- Piano di Qualifica (v2.0.0)
- Verbali interni:
- Verbali esterni:

2 Tecnologie utilizzate

La scelta delle teccnologie è stata guidata dalla necessità di rispetta i vincoli richiesti dalla proponente legati ad autoconsistenza, portabilità e facilità d'uso, nonché da considerazioni relative alla facilità di sviluppo, alla disponibilità di risorse e alla compatibilità con i requisiti funzionali e non funzionali del sistema.

3 Linguaggio di Programmazione: TypeScript

Scelta	Versione	Motivazione
TypeScript	5.9.2	Scelto per la sua capacità di fornire un sistema di tipi statico che migliora la sicurezza del codice e facilita la manutenzione, riducendo il rischio di errori comuni durante lo sviluppo. Inoltre, TypeScript è ampiamente supportato da un ecosistema ricco di librerie e strumenti, e si integra perfettamente con tecnologie come Angular ed Electron.

Tabella 2: Dettagli sulla scelta di TypeScript come linguaggio di programmazione

3.1 Deployment: Electron

Scelta	Versione	Motivazione
Electron	30.0.0	Scelto per la sua capacità di consentire lo sviluppo di applicazioni desktop multipiattaforma utilizzando tecnologie web, garantendo così la portabilità del prodotto su diversi sistemi operativi. Electron permette di creare un'esperienza utente nativa, sfruttando le potenzialità di Node.js per l'accesso al filesystem e alle risorse locali, facilitando l'integrazione con librerie e strumenti di terze parti.

Tabella 4: Dettagli sulla scelta di Electron come piattaforma di deployment

3.2 Framework Frontend: Angular

Scelta	Versione	Motivazione
Angular	21.2.5	Scelto per la sua capacità di supportare lo sviluppo di applicazioni web complesse e scalabili, con un'architettura basata su componenti che favorisce la modularità e la manutenibilità del codice. Inoltre, Angular offre un'ampia gamma di strumenti e funzionalità integrate, come la gestione dello stato, il routing e il supporto per la creazione di interfacce utente reattive, che lo rendono adatto a soddisfare i requisiti funzionali del progetto. La sua implementazione della Dependency Injection facilita la gestione delle dipendenze tra i componenti, migliorando la testabilità e la manutenibilità del codice.

Tabella 6: Dettagli sulla scelta di Angular come framework frontend

3.3 Librerie Aggiuntive

Scelta	Versione	Motivazione
Better SQLite	12.8.0	SQLite è un database relazionale che non richiede un server separato e può essere integrato direttamente nell'applicazione, facilitando lo sviluppo e il deployment. La sua leggerezza e portabilità lo rendono adatto a un'ampia gamma di piattaforme, garantendo la compatibilità con i requisiti di portabilità del progetto.
sqlite-vss	0.1.2	Scelto per la sua capacità di fornire una versione virtuale di SQLite, che consente di eseguire query su dati in memoria senza la necessità di creare un database fisico dei vettori per la ricerca semantica, semplificando così l'integrazione con il sistema.
ONNX Runtime	1.17.3	Scelto per la sua capacità di eseguire modelli di machine learning in modo efficiente e flessibile, supportando una vasta gamma di modelli e framework, facilitando l'integrazione di funzionalità di intelligenza artificiale nel prodotto.
Transformers.js	3.8.1	Scelto per la sua capacità di fornire un'interfaccia JavaScript per l'utilizzo di modelli di trasformatori, facilitando l'integrazione di funzionalità avanzate di elaborazione del linguaggio naturale e machine learning nel prodotto.

Tabella 8: Dettagli sulle scelte di librerie aggiuntive

3.4 Tecnologie di analisi statica

Scelta	Versione	Motivazione
SonarQube	2026.2	Scelto per la sua capacità di identificare e segnalare pattern di codice problematici, migliorando la qualità del codice e facilitando la manutenzione del progetto. Integrato nella pipeline di sviluppo, SonarQube consente di monitorare costantemente la qualità del codice, identificando potenziali bug, vulnerabilità e problemi di manutenzione, contribuendo così a garantire un prodotto finale robusto e affidabile.
Prettier	2.9.3	Scelto per la sua capacità di formattare automaticamente il codice in modo coerente, migliorando la leggibilità e facilitando la collaborazione tra i membri del team. Prettier supporta una vasta gamma di linguaggi e può essere facilmente integrato con editor di codice e strumenti di sviluppo, semplificando il processo di mantenimento di uno stile di codifica uniforme.

Tabella 10: Dettagli sulle scelte di tecnologie di analisi statica

3.5 Tecnologie di analisi dinamica

Scelta	Versione	Motivazione
Vitest	2.3.1	Scelto per la sua capacità di fornire un ambiente di test per applicazioni JavaScript e per il suo controllo dei componenti di Mock.
Playwright	1.59.1	Scelto per la sua capacità di fornire un ambiente di test per applicazioni web, con possibilità di testare flussi completi di interazione utente. Utilizzato quindi per i test E2E.

Tabella 12: Dettagli sulle scelte di tecnologie di analisi dinamica

4 Contesto del sistema

I vincoli di progetto, come specificato nell'Analisi dei Requisiti, richiedono che il sistema sia portabile, facile da usare e autoconsistente. Il caso d'uso principale del sistema è quello di un utente che desidera esplorare e interrogare un DIP per estrarre documenti fiscali e non, potendone verificare la validità e autenticità.

4.1 Modello architetturale

I vincoli di progetto hanno guidato la progettazione del sistema, influenzando le scelte tecnologiche e architetturali. Il sistema progettato quindi è pensato per non utilizzare tecnologie che richiedano installazioni o configurazioni complesse da parte dell'utente. I servizi offerti sono tutti accessibili in assenza di connettività, complesa la ricerca semantica.
Gli unici attori esterni coinvolti sono il file *system locale* da cui viene letto il DIP. STAMPA DA VEDERE SE HA SENSO METTERLO

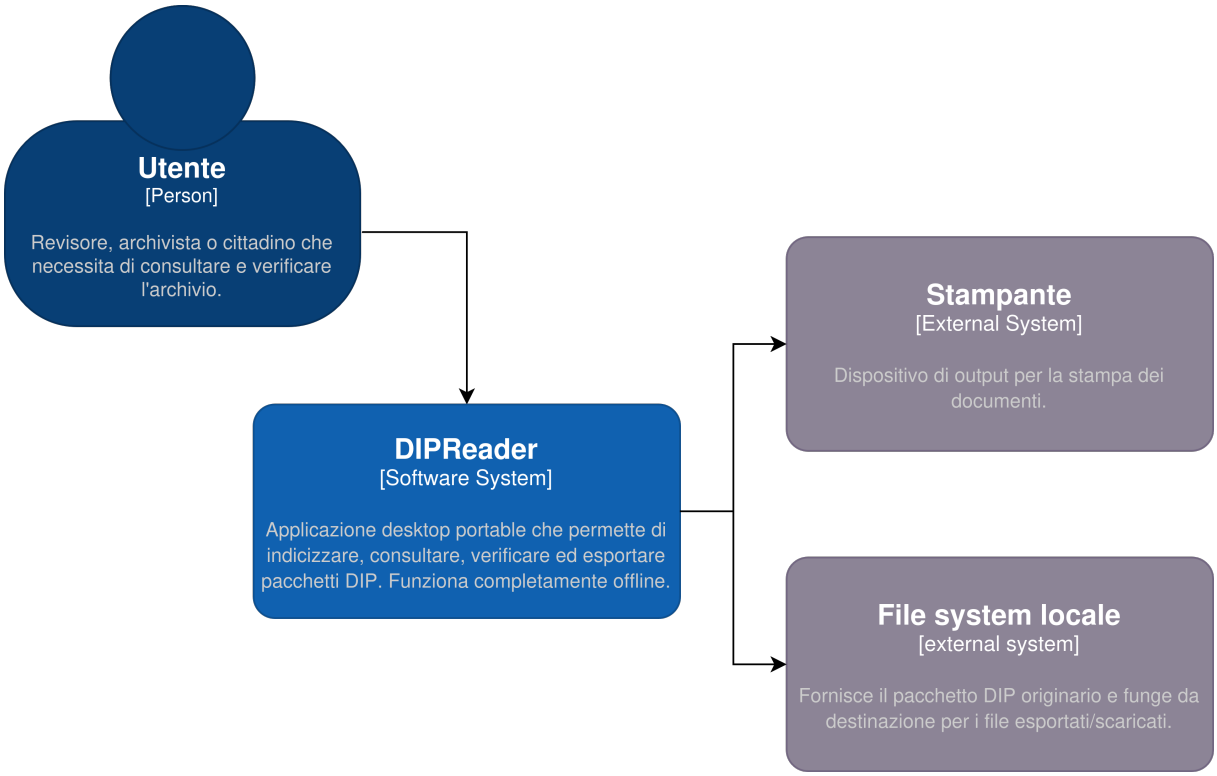

Figura 1: Diagramma C1 - Contesto del sistema *DIPReader*

4.2 Descrizione degli attori esterni

Gli unici attori esterni come già detto sono:

- il file system locale da cui viene letto il DIP
- STAMPA DA VEDERE SE HA SENSO METTERLO

5 Architettura dei container

La progettazione del sistema *DIPReader* è stata guidata dai principi di modularità, scalabilità e manutenibilità, con l'obiettivo di creare un'architettura che possa evolversi nel tempo e adattarsi a nuove esigenze e normative AGID.
In particolare, è stata completamente scorporata la logica di business dalla logica di presentazione, esponendo come unico punto di accesso comune gli *IPC channel*.
Il sistema così facendo permette di avere un backend facilmente estendibile, modificabile e soprattutto che si adatta allla struttura del DIP autonomamente. D'altra parte il frontend è completamente disaccoppiato ma strutturato in modo che i componenti di presentazione siano facilmente sostituibili, modificabili e soprattutto adattabili a nuove esigenze.

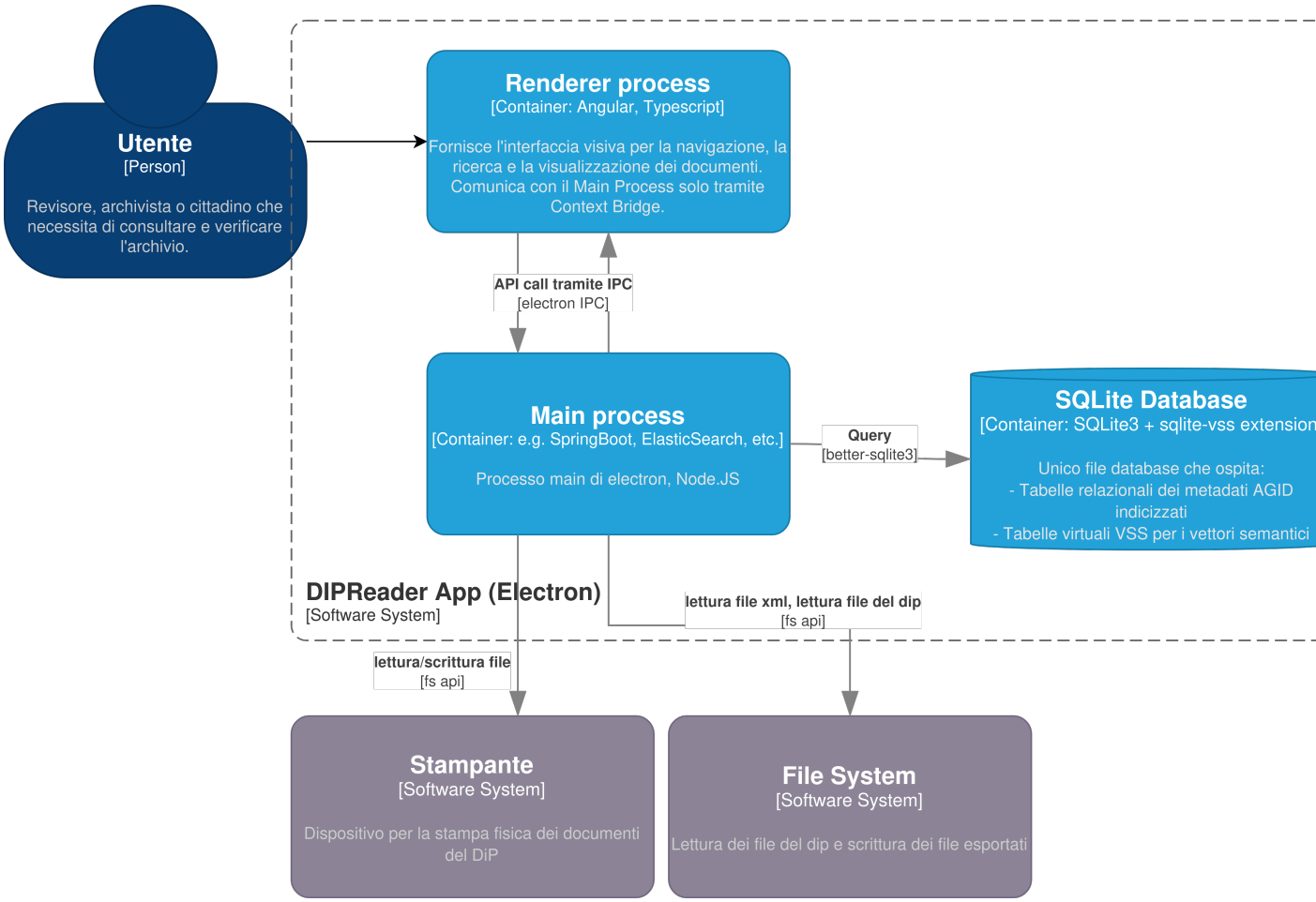

Figura 2: Diagramma C2 - Container del sistema *DIPReader*

5.1 Renderer

5.1.1 Struttura architetturale

Il container *Renderer* è responsabile della presentazione dell'interfaccia utente e dell'interazione con l'utente. Per la gestione quindi dell'interfaccia è stato scelto di strutturare il container per componenti. Ogni componente a sua volta è strutturato da una serie di services che si occupano di gestire la logica di presentazione, e da una serie di componenti di presentazione che si occupano di mostrare i dati all'utente.
Visto poi che le interazioni tra i diversi componenti di presentazione sono limitate, è stato scelto di strutturare il container in modo che ogni componente sia completamente disaccoppiato dagli altri. Gli unici punti di contatto risultano infatti essere il sistema di routing e i canali IPC, che permettono ai componenti di comunicare tra loro e con il backend.
In presenza poi di componenti come per esempio l'esportazione dei file, che richiedono di essere inseriti in più punti del sistema, è stato scelto di strutturare il container in modo che questi componenti siano facilmente integrabili in qualsiasi punto del sistema, senza dover modificare la struttura del container o dei componenti di presentazione grazie a delle semplici Injection di dipendenze.
È poi stato applicato il pattern Smart and Dumb Components, che permette di separare la logica di presentazione pura dalla logica di business che la coordina.

5.2 Main

5.2.1 Ruolo architetturale

Il container *Main* ospita la logica backend e costituisce il punto di orchestrazione tra IPC, use case applicativi e persistenza su SQLite. Le responsabilita principali sono la registrazione degli handler IPC, la risoluzione delle dipendenze tramite container DI, il coordinamento dei repository/DAO e il bootstrap iniziale dei dati indicizzati.

5.2.2 Stack tecnologico

Componente	**Tecnologia**
Runtime desktop	Electron (main + renderer)
Backend core	TypeScript
Dependency Injection	tsyringe
Database	better-sqlite3
IPC frontend-backend	ipcMain.handle / ipcRenderer.invoke
Architettura	Clean Architecture (layering)

5.2.3 Layer architetturali

1. **IPC Adapter:** boundary verso renderer, gestione canali, mapping DTO.
2. **Use Case:** thin orchestration layer, delega diretta ai repository.
3. **Repository:** astrazione accesso dati, delega ai DAO.
4. **DAO:** query SQL concrete, ricostruzione entity da righe.
5. **Mapper:** trasformazione entity $\leftrightarrow$ DTO.
6. **Entity / Value Objects:** modello di dominio.

5.2.4 Sequenza runtime generale

1. **Renderer invoca IPC:** `window.electronAPI.invoke(channel, payload)`.
2. **Preload autorizza:** verifica prefissi consentiti e inoltra a `ipcRenderer.invoke`.
3. **Main process intercetta:** adapter registra e gestisce l'handler del canale.
4. **Risoluzione DI:** l'adapter risolve il use case richiesto dal container.
5. **Use case delega:** orchestration verso il repository.
6. **Repository delega:** accesso ai dati tramite DAO.
7. **DAO esegue query:** interrogazione SQLite e ricostruzione entity.
8. **Conversione DTO:** mapping entity $\rightarrow$ DTO al boundary IPC.
9. **Risposta IPC:** il renderer riceve DTO o `null`.

5.2.5 Dependency Injection

Configurazione in `container.ts`:

1. **Provider DB:** `DATABASE_PROVIDER_TOKEN` $\rightarrow$ `DatabaseProvider`.
2. **Repository tokens:** `*_REPOSITORY_TOKEN` per i vari aggregati.
3. **DAO tokens:** `*_DAO_TOKEN` per le implementazioni concrete.
4. **Use case tokens:** `GET_BY_*` per le operazioni di retrieval.

5.2.6 Bootstrap e indicizzazione

DatabaseBootstrap.ts (avvio):

1. Ricrea database zero.
2. Applica schema.
3. Chiama `IndexDip.execute(dipPath)`.

IndexDip.execute (popolamento DB):

1. Insert: dip.
2. Insert: classi documentali.
3. Insert: processi (con metadati ricorsivi).
4. Insert: documenti (con metadati ricorsivi).
5. Insert: file.

5.2.7 Wiring runtime e registrazioni

In avvio applicazione (`main.ts`) il container runtime esegue:

1. bootstrap database tramite `ApplicationBootstrapAdapter`
2. registrazione token runtime `DIP_PATH_TOKEN`
3. registrazione adapter IPC (es. `BrowsingIpcAdapter`, `CheckIntegrityIpcAdapter`)

Nel bootstrap DB (`DatabaseBootstrap.ts`) viene registrato anche `SQLITE_DB_TOKEN` con la connessione SQLite runtime.

5.2.8 Semantica transazionale condivisa

Le operazioni backend che aggiornano alberi di dati su piu livelli usano `ITransactionManager` con implementazione SQLite (`SqliteTransactionManager`):

1. `BEGIN IMMEDIATE TRANSACTION`
2. esecuzione blocco applicativo
3. `COMMIT` su successo
4. `ROLLBACK` su errore

Questa semantica garantisce consistenza dei dati durante propagazioni gerarchiche di stato.

5.2.9 Sicurezza preload

`preload.ts` consente IPC su prefissi autorizzati: `ipc:search:`, `browse:`, `file:`, `check-integrity:`, `ipc:indexing:status`.

6 Architettura delle componenti

6.1 [Componente 1]

6.1.1 Descrizione

6.1.2 Interfacce

6.1.3 Dipendenze

6.1.4 Tracciamento dei requisiti

6.2 [Componente 2]

6.2.1 Descrizione

6.2.2 Interfacce

6.2.3 Dipendenze

6.2.4 Tracciamento dei requisiti

6.3 Componenti di Ricerca (FrontEnd)

6.3.1 Descrizione

I componenti di ricerca sono responsabili della gestione delle funzionalità di compilazione dei campi di ricerca.
Come già anticipato il pattern applicato è quello di Smart and Dumb Components, per cui i componenti di ricerca sono suddivisi in componenti di pura presentazione e gestiti da i componenti *"Padri"*, che si occupano di gestire la logica di questi ultimi nonchè le interazioni con altri servizi come la validazione dei filtri, la gestione dei campi di ricerca dinamici e il routing della sezione dei risultati. Nel contesto della ricerca questo coomponente Padre è SearchPageComponent.
Come si può notare questo componente sfrutta la dependency injection nativa di Angular per risolvere le dipendenze. Inoltre sono iniettate le interfaccie dei servizi necessari per il funzionamento e non gli effettivi servizi, applicando il principio di sostituzione di Liskov.

7 Backend - Browsing

7.1 Panoramica

Navigazione gerarchica

Il modulo Browsing implementa la navigazione gerarchica dei dati indicizzati in SQLite: $DIP_G \rightarrow$ Classi documentali $\rightarrow$ Processi $\rightarrow$ Documenti $\rightarrow$ File. Supporta inoltre getter alternativi per ID e stato di integrità.

Le caratteristiche backend trasversali al container Main (stack tecnologico, layering, sequenza runtime generale, Dependency Injection, bootstrap e sicurezza preload) sono descritte in Main.

7.1.1 Organizzazione file backend

Layer	File
Entry point IPC	`main.ts`, `preload.ts`, `ipc-channels.ts`,`BrowsingIpcAdapter.ts`
Use case	`dip/*`, `classe-documentale/*`,`process/*`, `document/*`, `file/*`
Repository	`I*Repository.ts`, `*Repository.ts`
DAO	`I*DAO.ts`, `*DAO.ts`, `mappers/*Mapper.ts`
Entity / DTO	`entity/*`, `dto/*`, `value-objects/*`
Database	`schema.sql`, `DatabaseBootstrap.ts`, `IndexDip.ts`

7.1.2 Flusso end-to-end

Flusso gerarchico di navigazione.

BROWSE_GET_DIP_BY_ID
↓
BROWSE_GET_DOCUMENT_CLASS_BY_DIP_ID
↓
BROWSE_GET_PROCESS_BY_DOCUMENT_CLASS
↓
BROWSE_GET_DOCUMENTS_BY_PROCESS
↓
BROWSE_GET_FILE_BY_DOCUMENT

Contratto IPC definito in `ipc-channels.ts` e registrato in `BrowsingIpcAdapter.ts`:

Canale	Payload	Return	Use case
BROWSE_GET_DOCUMENT_BY_ID	id: number	DocumentDTO | null	IGetDocumentByIdUC
BROWSE_GET_DOCUMENTS_BY_PROCESS	processId: number	DocumentDTO[]	IGetDocumentByProcessUC
BROWSE_GET_DOCUMENTS_BY_STATUS	status: IntegrityStatusEnum	DocumentDTO[]	IGetDocumentByStatusUC
BROWSE_GET_FILE_BY_ID	id: number	FileDTO | null	IGetFileByIdUC
BROWSE_GET_FILE_BUFFER_BY_ID	id: number	Buffer | null	IGetFileByIdUC + lettura fs
BROWSE_GET_FILE_BY_DOCUMENT	documentId: number	FileDTO[]	IGetFileByDocumentUC
BROWSE_GET_FILE_BY_STATUS	status: IntegrityStatusEnum	FileDTO[]	IGetFileByStatusUC
BROWSE_GET_PROCESS_BY_ID	id: number	ProcessDTO | null	IGetProcessByIdUC
BROWSE_GET_PROCESS_BY_STATUS	status: IntegrityStatusEnum	ProcessDTO[]	IGetProcessByStatusUC
BROWSE_GET_PROCESS_BY_DOCUMENT_CLASS	documentClassId: number	ProcessDTO[]	IGetProcessByDocumentClassUC
BROWSE_GET_DOCUMENT_CLASS_BY_DIP_ID	dipId: number	DocumentClassDTO[]	IGetDocumentClassByDipIdUC
BROWSE_GET_DOCUMENT_CLASS_BY_STATUS	status: IntegrityStatusEnum	DocumentClassDTO[]	IGetDocumentClassByStatusUC
BROWSE_GET_DOCUMENT_CLASS_BY_ID	id: number	DocumentClassDTO | null	IGetDocumentClassByIdUC
BROWSE_GET_DIP_BY_ID	id: number	DipDTO | null	IGetDipByIdUC
BROWSE_GET_DIP_BY_STATUS	status: IntegrityStatusEnum	DipDTO[]	IGetDipByStatusUC

Tabella 15: Contratto IPC del modulo Browsing

Nota: Il canale `BROWSE_GET_DIP_BY_DOCUMENT_CLASS` è definito in `ipc-channels.ts` ma non registrato in `BrowsingIpcAdapter`.

7.1.3 Use Case: firme getter

Caratteristica della layer

Gli use case sono thin wrapper senza logica aggiuntiva; delegano direttamente al repository corrispondente.

Use Case	Firma
IGetDipByIdUC	execute(id: number): Dip | null
IGetDipByStatusUC	execute(status: IntegrityStatusEnum): Dip[]
IGetDocumentClassByDipIdUC	execute(dipId: number): DocumentClass[]
IGetDocumentClassByIdUC	execute(id: number): DocumentClass | null
IGetDocumentClassByStatusUC	execute(status: IntegrityStatusEnum): DocumentClass[]
IGetProcessByDocumentClassUC	execute(documentClassId: number): Process[]
IGetProcessByIdUC	execute(id: number): Process | null
IGetProcessByStatusUC	execute(status: IntegrityStatusEnum): Process[]
IGetDocumentByIdUC	execute(id: number): Document | null
IGetDocumentByProcessUC	execute(processId: number): Document[]
IGetDocumentByStatusUC	execute(status: IntegrityStatusEnum): Document[]
IGetFileByIdUC	execute(id: number): File | null
IGetFileByDocumentUC	execute(documentId: number): File[]
IGetFileByStatusUC	execute(status: IntegrityStatusEnum): File[]

Interfacce per aggregato.

7.1.4 Layer Repository

Ogni repository delega al DAO corrispondente il retrieval dei dati.

Repository	Implementazione	Metodi getter
IDipRepository	DipRepository.ts	ById, ByUuid, ByStatus
IDocumentClassRepository	DocumentClassRepository.ts	ById, ByDipId, ByStatus
IProcessRepository	ProcessRepository.ts	ById, ByDocumentClassId, ByStatus
IDocumentRepository	DocumentRepository.ts	ById, ByProcessId, ByStatus
IFileRepository	FileRepository.ts	ById, ByDocumentId, ByStatus

Inoltre: `save`, `updateIntegrityStatus` per persistenza; `searchDocument*`, `getIndexedDocumentsCount` per DocumentRepository.

7.1.5 Layer DAO

Le interfacce `I*DAO` espongono metodi paralleli ai repository per il retrieval e query concrete eseguite in SQLite.

DAO	File	Query / Operazioni
`DipDAO`	`DipDAO.ts`	By id/uuid/status; UPSERT uuid
`DocumentClassDAO`	`DocumentClassDAO.ts`	By id/dipId/status; search LIKE; UPSERT
`ProcessDAO`	`ProcessDAO.ts`	By id/documentClassId/status; metadati albero
`DocumentDAO`	`DocumentDAO.ts`	By id/processId/status; search avanzate; ricerca vettoriale
`FileDAO`	`FileDAO.ts`	By id/documentId/status; sort is_main DESC
`MetadataHelper`	`MetadataHelper.ts`	Load/save ricorsivo; ricostruzione albero

7.1.6 Entity e Value Objects

Modello di dominio. Ogni entity espone getter e `setIntegrityStatus`:

Entity	Attributi principali
`Dip`	`id,uuid,integrityStatus`
`DocumentClass`	`id,dipId,dipUuid,uuid,name,timestamp,integrityStatus`
`Process`	`id,documentClassId,documentClassUuid,uuid,metadata, integrityStatus`
`Document`	`id,uuid,metadata,processId,processUuid,integrityStatus`
`File`	`id,uuid,filename,path,hash,isMain,documentId, documentUuid,integrityStatus`

7.1.7 Data Transfer Objects

Contratto dati IPC (restituiti dall'adapter al renderer):

DTO	Attributi
`DipDTO`	`id,uuid,integrityStatus`
`DocumentClassDTO`	`id,dipId,uuid,name,timestamp,integrityStatus`
`ProcessDTO`	`id,documentClassId,uuid,integrityStatus,metadata: MetadataDTO`
`DocumentDTO`	`id,processId,uuid,integrityStatus,metadata: MetadataDTO`
`FileDTO`	`id,documentId,filename,path,hash,integrityStatus, isMain`
`MetadataDTO`	`name,value (scalare|MetadataDTO[]),type: MetadataType`

Tabella 21: DTO schema (metadati caricati ricorsivamente)

Nota: Il converter genera eccezioni se `id` o FK sono `null` durante la conversione.

7.1.8 Mapping e conversioni

Trasformazioni tra layer di persistenza, entity e DTO:

Conversione	File	Funzioni
Entity → DTO	`EntityToDtoConverter.ts`	`*ToDto` per dip, docClass, process, document, file, metadata
SQL → Entity	`*Mapper.ts`	`*Mapper` per dip, docClass, process, document, file, metadata

`MetadataMapper.fromFlatRows` ricostruisce l'albero da righe flat con controllo cicli/root.

7.1.9 Comportamenti runtime

Semantica getter

By-ID: restituisce `null` se entity non trovata
List: restituisce array, anche vuoto
DTO: conversione applicata nel boundary IPC prima della restituzione

Lettura file buffer (`BROWSE_GET_FILE_BUFFER_BY_ID`).

1. Recupera file entity tramite `IGetFileByIdUC`
2. Risolve percorso assoluto: `path.resolve(dipPath, file.getPath())`
3. Legge contenuto con `fs.readFileSync`
4. In caso di errore I/O: log e ritorno `null`

7.1.10 Test coverage

Suite	Copertura
`BrowsingIpcAdapter.spec.ts`	Canali; mapping DTO; semantica null
`*UseCases.spec.ts`	Use case → repository
`*Repository.spec.ts`	Repository → DAO
`*DAO.spec.ts`	Query SQL; persistenza; ricostruzione entity

8 Backend - Verifica Integrita

8.1 Panoramica

Verifica gerarchica dello stato

Il modulo Verifica Integrita esegue controlli su 5 livelli gerarchici: File → Documento → Processo → Classe documentale → DIP. Il controllo puo essere avviato da qualsiasi livello e propaga lo stato nel sottoalbero interessato.

8.1.1 Perimetro funzionale

La verifica opera sui seguenti livelli:

1. **File**
2. **Documento**
3. **Processo**
4. **Classe documentale**
5. **DIP**

Ogni invocazione calcola lo stato del nodo root e dei suoi discendenti, con persistenza progressiva in SQLite tramite `updateIntegrityStatus(...)`.

8.1.2 Organizzazione file backend

Layer	File
Entry point IPC	`main.ts`, `preload.ts`, `ipc-channels.ts`, `CheckIntegrityIpcAdapter.ts`
Use case	`*/ICheck*IntegrityStatusUC.ts`, `*/impl/Check*IntegrityStatusUC.ts`
Service	`IntegrityVerificationService.ts`, `HashingService.ts`
Transaction manager	`SqliteTransactionManager.ts`
Repository	`I*Repository.ts`, `*Repository.ts`
DAO	`I*DAO.ts`, `*DAO.ts`
Entity / DTO	`entity/*`, `dto/*`
Database	`schema.sql` (colonna `integrity_status` su 5 tabelle)

Tabella 25: Organizzazione modulo Verifica Integrita

8.1.3 Flusso end-to-end

Sequenza di verifica.

1. Il renderer invoca il canale `check-integrity:*`
2. `CheckIntegrityIpcAdapter` intercetta l'handler e risolve il use case
3. Il use case delega a `IntegrityVerificationService`
4. Il service esegue `runInTransaction(...)`
5. Il service verifica hash file e aggrega stati sui parent
6. Il service persiste gli stati sui livelli coinvolti
7. L'handler IPC ritorna `IntegrityStatusEnum`

Flusso gerarchico di propagazione.

CHECK_DIP_INTEGRITY_STATUS
↓
CHECK_DOCUMENT_CLASS_INTEGRITY_STATUS
↓
CHECK_PROCESS_INTEGRITY_STATUS
↓
CHECK_DOCUMENT_INTEGRITY_STATUS
↓
CHECK_FILE_INTEGRITY_STATUS

Contratto IPC definito in `ipc-channels.ts` e registrato in `CheckIntegrityIpcAdapter.ts`:

Canale	Payload	Return	Use case
CHECK_FILE_INTEGRITY_STATUS	id: number	IntegrityStatusEnum	ICheckFileIntegrityStatusUC
CHECK_DOCUMENT_INTEGRITY_STATUS	id: number	IntegrityStatusEnum	ICheckDocumentIntegrityStatusUC
CHECK_PROCESS_INTEGRITY_STATUS	id: number	IntegrityStatusEnum	ICheckProcessIntegrityStatusUC
CHECK_DOCUMENT_CLASS_INTEGRITY_STATUS	id: number	IntegrityStatusEnum	ICheckDocumentClassIntegrityStatusUC
CHECK_DIP_INTEGRITY_STATUS	id: number	IntegrityStatusEnum	ICheckDipIntegrityStatusUC

Tabella 26: Contratto IPC del modulo Verifica Integrita

Nota: Il canale `INTEGRITY_VERIFY` e definito in `ipc-channels.ts` ma non risulta registrato in `CheckIntegrityIpcAdapter`.

8.1.4 Use Case: firme check

Caratteristica della layer

I use case sono thin wrapper senza logica aggiuntiva; delegano direttamente a `IntegrityVerificationService`.

Use Case	Firma
`ICheckFileIntegrityStatusUC`	`execute(fileId: number): Promise<IntegrityStatusEnum>`
`ICheckDocumentIntegrityStatusUC`	`execute(documentId: number): Promise<IntegrityStatusEnum>`
`ICheckProcessIntegrityStatusUC`	`execute(processId: number): Promise<IntegrityStatusEnum>`
`ICheckDocumentClassIntegrityStatusUC`	`execute(documentClassId: number): Promise<IntegrityStatusEnum>`
`ICheckDipIntegrityStatusUC`	`execute(dipId: number): Promise<IntegrityStatusEnum>`

Tabella 28: Firme use case Verifica Integrita

8.1.5 Servizio centrale

File: `IntegrityVerificationService.ts`.

Dipendenze iniettate.

1. `IFileRepository`
2. `IDocumentRepository`
3. `IProcessRepository`
4. `IDocumentClassRepository`
5. `IDipRepository`
6. `IHashingService`
7. `ITransactionManager`
8. `DIP_PATH_TOKEN`

Regole operative.

1. Ogni API pubblica gira dentro `runInTransaction`
2. Se la root non esiste, viene lanciato errore
3. La verifica e ricorsiva nel sottoalbero
4. Ogni livello aggiorna `integrity_status` via repository

8.1.6 Regole di aggregazione

Algoritmo aggregateStatuses

Caso 1: nessun figlio → `VALID`
Caso 2: almeno un figlio `INVALID` → `INVALID`
Caso 3: nessun `INVALID` ma almeno un UNKNOWN → UNKNOWN
Caso 4: tutti `VALID` → `VALID`

Metodo	Aggiornamenti
`checkFileIntegrityStatus`	Verifica hash file; aggiorna solo file
`checkDocumentIntegrityStatus`	Verifica file del documento; aggiorna file + documento
`checkProcessIntegrityStatus`	Verifica documenti e file del processo; aggiorna file + documento + processo
`checkDocumentClassIntegrityStatus`	Verifica processi/documenti/file della classe; aggiorna livelli inferiori + classe
`checkDipIntegrityStatus`	Verifica intero sottoalbero; aggiorna file + documento + processo + classe + DIP

Tabella 30: Semantica operativa per livello

Semantica per livello.

8.1.7 Hashing e transazioni nel modulo

HashingService. `checkFileIntegrity(filePath, expectedHash)` legge bytes, calcola SHA-256, converte digest Base64 e confronta con l'hash atteso.

Path assoluto file. Il service risolve il percorso con `path.resolve(dipPath, file.getPath())` prima della verifica hash.

Transaction manager. L'esecuzione avviene in `runInTransaction(work)`; in caso di errore durante la cascata e garantito rollback completo.

8.1.8 Layer Repository e DAO

Componente	Metodi rilevanti per verifica
`IFileRepository / FileDAO`	`getById`, `getByDocumentId`, `updateIntegrityStatus`
`IDocumentRepository / DocumentDAO`	`getById`, `getByProcessId`, `updateIntegrityStatus`
`IProcessRepository / ProcessDAO`	`getById`, `getByDocumentClassId`, `updateIntegrityStatus`
`IDocumentClassRepository / DocumentClassDAO`	`getById`, `getByDipId`, `updateIntegrityStatus`
`IDipRepository / DipDAO`	`getById`, `updateIntegrityStatus`

Tabella 32: Repository e DAO coinvolti nella verifica integrita

8.1.9 Comportamenti runtime

Semantica runtime

Root non trovato: eccezione su id non presente
Persistenza: aggiornamento `integrity_status` su tutti i livelli coinvolti
Atomicita: rollback completo su errore interno
Return IPC: stato finale `IntegrityStatusEnum`

8.1.10 Test coverage

Suite	Copertura
`CheckIntegrityIpcAdapter.spec.ts`	Registrazione canali; forwarding payload; propagazione risultato/errore
`*UseCases.spec.ts`	Delega use case → service; propagazione errori
`IntegrityVerificationService.spec.ts`	Not found su root; cascata stati; propagazione invalid; caso senza figli
`IntegrityVerificationService.integration.spec.ts`	Persistenza in-memory; rollback completo su errore hashing; validazione stati su tutte le tabelle
`HashingService.spec.ts`, `SqliteTransactionManager.test.ts`	Hash valido/non valido; begin/commit e begin/rollback

Tabella 34: Copertura test modulo Verifica Integrita